

Architecture et Programmation Web

Séance 9 : Introduction aux API REST

Nicolas Burny

23 avril 2026

ECAM

Le concept d'API

Qu'est-ce qu'une API ?

- **API** : Application Programming Interface. Interface logicielle permettant à deux applications de communiquer de manière structurée.
- **Abstraction** : Permet d'utiliser les fonctionnalités d'un programme sans connaître son implémentation interne.
- **Contrat technique** : Définit précisément les requêtes possibles, les formats de données et les réponses attendues.

- **Avant** : L'application Flask génère tout le HTML (Monolithe).
- **Aujourd'hui** : Le backend ne fournit que de la **donnée brute** (API).
- **Bénéfices** :
 - Multiples clients (Mobile, Web SPA, IoT).
 - Séparation nette des responsabilités.

L'Architecture REST

- Proposé par Roy Fielding en 2000.
- **Ressources** : Tout est une ressource identifiable par une URI.
- **Stateless** : Pas d'état conservé entre les requêtes API (chaque requête est complète).
- **Uniform Interface** : Utilisation standard des verbes HTTP.

- **GET** : Lire des données (sans effet de bord).
- **POST** : Créer une nouvelle ressource.
- **PUT** : Remplacer **complètement** une ressource existante.
- **PATCH** : Modifier **partiellement** une ressource existante.
- **DELETE** : Supprimer une ressource.

Exemple URI

GET /api/features → Liste des demandes.

POST /api/features → Ajout d'une demande.

Conception d'une API REST

- **Utiliser des noms** : Les verbes sont portés par la méthode HTTP.
 - À éviter : GET /getAllFeatures ou POST /create_feature.
 - Recommandé : GET /features ou POST /features.
- **Pluriel par convention** : /users plutôt que /user.
- **Hiérarchie logique** : /features/42/comments pour les ressources liées.

- **Méthode Sûre** : Ne modifie pas l'état du serveur (ex : GET).
- **Idempotence** : L'exécution répétée d'une requête a le même effet sur le serveur qu'une exécution unique.

Méthode	Sûre	Idempotente
GET	✓	✓
POST	✗	✗
PUT	✗	✓
PATCH	✗	✗
DELETE	✗	✓

Le filtrage permet d'extraire des sous-ensembles de données sans multiplier les points d'entrée (endpoints).

- Utilisation des **Query Parameters** (après le ?).
- **Exemple simple** : `GET /api/features?priority=5`
- **Filtres multiples** : `GET /api/features?status=open&priority=5`

Bonne pratique

Ne pas utiliser le filtrage pour des ressources hiérarchiques (ex : préférez `/users/1/posts` à `/posts?user_id=1`).

Essentielle pour la performance et l'expérience utilisateur lorsque le volume de données est important.

- **Approche par page** : `/api/features?page=2&per_page=10`
- **Approche par curseur (Offset)** : `/api/features?limit=20&offset=40`

Métadonnées de réponse

Il est d'usage de renvoyer le nombre total d'éléments dans les headers ou dans un objet enveloppe : `{ "total": 150, "data": [...] }`

Permet au client de définir l'ordre de présentation des ressources.

- **Paramètres types** : `sort` (champ) et `order` (direction).
- **Exemple** : `GET /api/features?sort=title&order=asc`
- **Syntaxe compacte** : `GET /api/features?sort=-created_at` (le - indique un tri descendant).

Versioning d'une API

Une API publique évolue. Le versioning évite de briser les clients existants lors des changements.

- **URL path** (recommandé) : `/api/v1/features`
- **Query parameter** : `/api/features?version=1`
- **Header** : `Accept: application/vnd.app.v1+json`

Bonne pratique

L'approche **URL path** est la plus visible et la plus simple à tester (navigateur, curl). Commencer à versionner dès le début évite les migrations douloureuses.

Flask facilite l'accès aux paramètres de requête via l'objet `request.args`.

- `request.args` est un dictionnaire contenant les Query Parameters.

Exemple de code

```
@app.route('/api/features')
def get_features():
    priority = request.args.get('priority', type=int)
    page = request.args.get('page', default=1, type=int)
    # Logique de filtrage en base de donnees...
    return jsonify(results)
```

Implémentation avec Flask

Anatomie d'une Requête HTTP

`POST /api/features?draft=true` <- méthode + chemin + query param

`Host: localhost:5000`

`Content-Type: application/json` <- headers

`Authorization: Bearer eyJhbG...`

`{` <- body (POST / PUT / PATCH uniquement)

`"title": "Dark Mode",`

`"priority": 3`

`}`

- **Path params** (/features/{id}) : identifient une ressource précise.
- **Query params** (?page=1) : filtrent ou configurent la réponse.
- **Headers** : métadonnées (authentification, format des données).
- **Body** : données envoyées — uniquement pour POST / PUT / PATCH.

Créer un endpoint JSON

On utilise `jsonify` pour sérialiser les données.

```
from flask import jsonify

@app.route('/api/features/<int:id>')
def get_feature(id):
    feature = FeatureRequest.query.get_or_404(id)
    return jsonify({
        "id": feature.id,
        "title": feature.title,
        "priority": feature.priority
    })
```

Code	Nom	Usage type
200	OK	Succès général (GET, PUT, PATCH).
201	Created	Ressource créée (POST).
204	No Content	Succès sans corps (DELETE).
400	Bad Request	JSON malformé, champ manquant.
401	Unauthorized	Non authentifié.
403	Forbidden	Authentifié mais non autorisé.
404	Not Found	Ressource introuvable.
409	Conflict	Doublon (ex : email déjà utilisé).
422	Unprocessable	Format OK, valeur invalide.
500	Server Error	Erreur interne imprévue.

Structurer les Réponses d'Erreur

Une API doit retourner des erreurs **lisibles** par la machine et **cohérentes**.

Format recommandé

```
{  
  "error": "Le titre est requis.",  
  "field": "title",  
  "code": 422  
}
```

- Toujours renvoyer du JSON, même pour les erreurs.
- Inclure un message **humainement lisible** et un code machine.
- La **RFC 7807** (*Problem Details for HTTP APIs*) standardise ce format.

Sécurité et Statelessness

- REST est par définition **sans état** (*stateless*).
- Le serveur ne stocke pas de contexte client entre les requêtes (pas de variable globale *session*).
- Chaque requête doit être autonome et contenir toutes les informations nécessaires à son traitement.
- **Bénéfices** : Scalabilité horizontale simplifiée et meilleure résilience du serveur.
- **Problème** : Comment gérer l'authentification ?

Sessions (Cookies)

- État stocké côté serveur (DB ou RAM).
- Très lié au navigateur.
- Difficile à partager entre domaines.

Tokens (Bearer)

- État stocké côté client.
- Adapté aux mobiles et services tiers.
- Facilite l'architecture micro-services.

Pourquoi Flask-Login n'est-il pas adapté ici ?

- **Basé sur les sessions** : flask-login s'appuie sur les cookies et la gestion de sessions de Flask.
- **Violation du Stateless** : Il maintient un état de connexion côté serveur, ce qui contredit le principe **stateless** des API REST.
- **Couplage fort au navigateur** : Pensé pour des applications web classiques, il est moins adapté aux clients diversifiés (mobiles, IoT, autres serveurs).
- **Solution pour les APIs** : Utiliser une authentification par **Tokens** (ex : JWT via Flask-JWT-Extended) transmis à chaque requête.

Authentification par Bearer Token

- Le client s'authentifie une fois et reçoit un jeton (ex : JWT).
- Ce jeton est transmis dans chaque requête via le Header HTTP.
- **Format standard :**

`Authorization: Bearer <votre_token>`

Exemple de requête cURL

```
curl -X GET https://api.app.com/v1/me \  
-H "Authorization: Bearer eyJhbGciOiJIUzI1..."
```

Flask-JWT-Extended : Configuration

```
pip install Flask-JWT-Extended
```

```
from flask_jwt_extended import JWTManager
```

```
app.config['JWT_SECRET_KEY'] = 'ma-cle-secrete'
```

```
jwt = JWTManager(app)
```

Clé secrète

La clé sert à **signer** les tokens. Ne jamais la committer dans le code source — utiliser une variable d'environnement en production.

Générer un Token (Route Login)

```
from flask_jwt_extended import create_access_token
from werkzeug.security import check_password_hash

@app.route('/api/login', methods=['POST'])
def login():
    data = request.get_json() or {}
    user = User.query.filter_by(username=data.get('username')).first()

    if not user or not check_password_hash(user.password_hash, data.get('password')):
        return jsonify({'error': 'Identifiants invalides'}), 401

    token = create_access_token(identity=user.username)
    return jsonify(access_token=token), 200
```

Le client stocke ce token et l'envoie dans chaque requête suivante.

Protéger une Route avec @jwt_required()

```
from flask_jwt_extended import jwt_required, get_jwt_identity

@app.route('/api/features/<int:id>', methods=['DELETE'])
@jwt_required()          # Retourne 401 automatiquement si token absent ou invalide
def delete_feature(id):
    current_user = get_jwt_identity() # Valeur passée à create_access_token()

    feature = FeatureRequest.query.get_or_404(id)
    db.session.delete(feature)
    db.session.commit()
    return '', 204
```

- @jwt_required() bloque la requête si le header Authorization: Bearer ... est absent ou invalide.
- get_jwt_identity() récupère l'identité sans toucher la base de données.

Structure d'un Token JWT

Un JWT est composé de trois parties encodées en Base64, séparées par des points :

`header.payload.signature`

Header

Algorithme utilisé.

```
{"alg": "HS256",  
  "typ": "JWT"}
```

Payload

Données (claims).

```
{"sub": "42",  
  "role": "admin"}
```

Signature

HMAC(header + payload,
clé secrète du serveur).

À retenir

Header et Payload sont **lisibles par tous**. Seule la signature garantit l'intégrité.

Sécurité et Intégrité des Tokens (JWT)

- **Sont-ils cryptés ?**
 - Généralement **non**. Ils sont **signés**.
 - Le contenu est simplement encodé en Base64 : il est lisible par n'importe qui.
 - *Règle d'or* : Ne jamais stocker d'informations sensibles (mots de passe) dans un token.
- **L'utilisateur peut-il les modifier ?**
 - Techniquement oui, mais cela invalide la **signature**.
 - Le serveur utilise une clé secrète pour vérifier que le contenu n'a pas été altéré.
 - Si un seul bit change, le serveur rejette le token.
- **Intérêt** : Permet au serveur de vérifier l'identité sans consulter sa base de données à chaque requête (*Stateless*).

- **Séparation des responsabilités** : Le backend devient un fournisseur de données brutes, indépendant de l'interface utilisateur.
- **Standardisation** : REST impose une structure prévisible via les ressources (URIs) et la sémantique des verbes HTTP.
- **Scalabilité** : Le mode *stateless* facilite la montée en charge, en déléguant la gestion de l'état au client via des tokens (JWT).
- **Interopérabilité** : Une API bien conçue permet de servir simultanément des clients Web, mobiles et tiers.
- **Flexibilité** : Les API REST sont adaptées à de nombreux langages et frameworks.

Questions ?